

6 EE PROJECTS THAT BUILD A REAL SATELLITE

Thrusters. Attitude control. Flight software. Ground stations. Power.
The actual subsystems of a spacecraft. Real EE.

1. Cold-Gas Reaction Thruster — slew a platform by firing gas	\$75
2. Auto-Tracking Ground Station — an antenna that follows satellites	\$70
3. Magnetorquer Detumble System — stabilize against Earth's field	\$30
4. Satellite-in-a-Box Flight Computer — live telemetry downlink	\$50
5. Sun-Vector Sensor — find the Sun for attitude	\$25
6. CubeSat Power System (EPS) — a satellite's electrical heart	\$45

PROJECT 1

Cold-Gas Reaction Thruster

A platform that rotates and holds its heading by firing puffs of compressed gas — exactly like a spacecraft's RCS

Cost: ~\$75

Difficulty: Advanced

Build Time: 2-3 weekends

Why This Matters

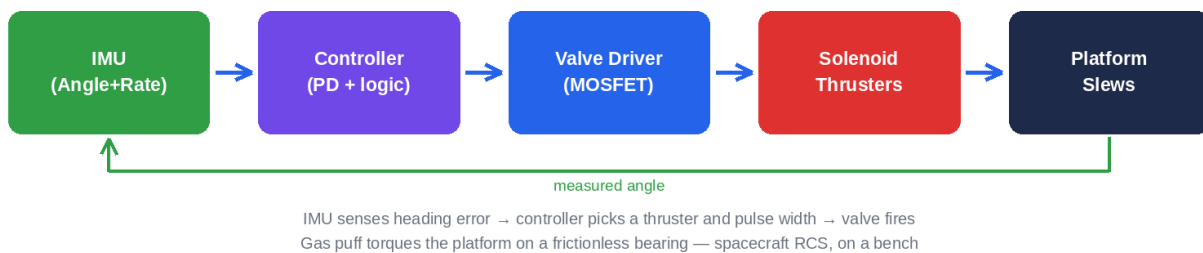
Out in orbit there is nothing to push against, so spacecraft change and hold their orientation by firing small thrusters — the reaction control system (RCS). You can build the same thing on the ground: mount a platform on a near-frictionless bearing, plumb a few small solenoid valves to a compressed-gas supply, and fire short puffs to rotate it and hold a commanded heading. An IMU senses the angle, a controller decides which thruster to pulse and for how long, and the platform slews and stops on target — no wheels, no contact, just gas. This is exactly how satellites maneuver, and the air-bearing version is literally how aerospace labs test real attitude-control hardware before flight. It is the most viscerally 'spacecraft' project you can build on a bench.

What this proves to a recruiter: You understand reaction-control propulsion, closed-loop attitude control, solenoid valve driving and pulse-width thrust modulation, and inertial sensing. Propulsion and GNC are core spacecraft disciplines, and demonstrating thruster-based attitude control on an air bearing is exactly the kind of hardware aerospace teams build and test.

→ **The job this maps to:** spacecraft propulsion and GNC — satellite and launch companies, and the air-bearing test labs that validate attitude-control systems.

Safety: Compressed gas is genuinely dangerous. Use only a properly rated regulator and tubing, never exceed component pressure ratings, and secure the gas supply so it cannot become a projectile. Use inert gas (CO₂ or filtered compressed air) — never anything flammable or an unregulated high-pressure source — wear eye protection, and fully depressurize before working on the plumbing.

Cold-Gas Reaction Thruster — Attitude Control Loop



How It Works

- **Newton's third law in a vacuum:** A spacecraft cannot push against anything, so it carries its own reaction mass: expel gas one way, the body rotates the other. Firing a thruster offset from the center of mass produces a torque that slews the vehicle. Your bench version does the identical thing on a low-friction bearing that mimics the freedom of orbit.
- **Near-frictionless bearing:** The platform sits on a ball bearing, a lazy-Susan, or — for the stretch version — a planar air bearing that floats it on a thin film of air. The lower the friction, the more the demo behaves like a real satellite, where the only torques are the ones you command.
- **Solenoid 'thrusters':** Small 12 V solenoid valves act as on/off thrusters. Mounted in opposing pairs at the platform's edge and fed from a regulated gas line, each releases a puff of gas through a nozzle when energized. Opposing pairs let you torque clockwise or counter-clockwise.
- **Pulsed thrust control:** You cannot throttle a solenoid, so you modulate thrust the way real RCS does — by pulse width and pulse frequency. Short pulses give fine control near the target; longer or more frequent pulses give a fast slew. The controller translates a commanded torque into a valve on-time.
- **Closed-loop attitude control:** An IMU measures the platform's angle and rate; a PD controller computes the torque needed to reach and hold the setpoint, then thruster-selection logic fires the correct valve. Because gas is limited, the controller must avoid a 'limit cycle' — chattering around the target and wasting gas — which is a real spacecraft design problem.

Key Design Decisions

- **Why cold gas instead of anything fancier?** Cold-gas propulsion — just releasing a pressurized inert gas — is the simplest, safest thruster there is, and it is genuinely flown on small satellites. No combustion, no high voltage, no toxic propellant: ideal for learning the control problem without the hazards of real rocketry.
- **Why pulse-width modulate thrust?** A solenoid valve is on or off; it cannot be partly open in a controllable way. Spacecraft RCS solves this with pulse modulation — varying how long and how often you fire — mapping a continuous torque command onto discrete puffs. Tuning it is the heart of the project.
- **Why an air bearing for testing?** Aerospace labs validate attitude-control hardware on air-bearing tables precisely because they remove friction and gravity torque, leaving the vehicle free to rotate as it would in orbit. Building even a simple version teaches you how flight hardware is actually tested on the ground.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Solenoid valves	12 V, small pneumatic (opposing pairs)	4	\$20
Gas supply + regulator	CO2 cartridge or small tank + regulator	1	\$22
Tubing + nozzles	Rated pneumatic tubing + fittings	1	\$8
IMU	MPU-6050 / BNO055	1	\$4
MOSFET drivers	Logic-level + flyback diodes	1	\$3
Microcontroller	ESP32 / STM32	1	\$6
Low-friction bearing	Lazy-Susan or air-bearing puck	1	\$8
Platform + battery	Rigid disk + LiPo	1	\$8

Build Steps

Build the frictionless platform: Mount a rigid disk on a low-friction bearing (a quality lazy-Susan to start, a planar air bearing for the stretch build). Balance it carefully so gravity introduces no torque — it should sit still at any angle when undisturbed.

Plumb the gas system: Fit a regulator to the gas supply and run rated tubing to opposing solenoid valves at the rim, each ending in a small nozzle. Keep supply pressure within every component's rating, and leak-check before powering anything.

Wire valves + IMU: Drive each solenoid from a MOSFET with a flyback diode (valves are inductive). Mount the IMU at the platform's center and read angle and rate over I2C. Confirm a brief valve pulse produces a clean puff and a small rotation.

Close the control loop: Implement a PD controller: from heading error and rate, compute a torque command, then convert it to a valve and a pulse width. Add a deadband so the platform does not fire constantly near the target — that is how you stop wasting gas in a limit cycle.

Slew and hold on camera: Command a 90-degree turn and film the platform puffing gas, rotating, and stopping precisely on the new heading. Nudge it and watch it fire to recover. Log angle vs. time to show the slew and settle — the language of attitude control.

Interview Questions:

- How does a reaction-control thruster change a spacecraft's orientation with no external reference?
- Why is thrust modulated by pulse width and frequency rather than throttling the valve?
- What is a limit cycle in RCS control, and why does it waste propellant?
- Why are air-bearing tables used to test attitude-control hardware on the ground?
- Why is cold-gas propulsion attractive for small satellites despite its low performance?
- How would you allocate thruster firings for combined slew and disturbance rejection?

PROJECT 2

Auto-Tracking Ground Station

A motorized antenna that locks onto a satellite and physically follows it across the sky

Cost: ~\$70

Difficulty: Advanced

Build Time: 2-3 weekends

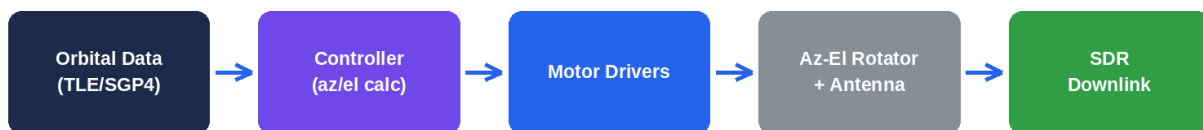
Why This Matters

A satellite in low orbit crosses the sky in minutes, so to keep a high-gain antenna pointed at it you need a tracker that moves. This project builds a real azimuth-elevation rotator: two motors aim an antenna, a microcontroller predicts the satellite's position from its orbital elements, and the antenna physically slews to follow the pass from horizon to horizon — exactly what a professional ground station does. Pair it with an SDR and you can hold a lock on a fast-moving satellite and pull down a clean signal a fixed antenna would lose in seconds. It is the ground half of every space mission, and watching your antenna hunt across the sky on its own is genuinely striking.

What this proves to a recruiter: You understand satellite tracking and orbital prediction (TLEs/SGP4), azimuth-elevation pointing and motor control, antenna gain and link geometry, and ground-station automation. The ground segment is a huge part of the space industry, and a working auto-tracker is a concrete TT&C; and RF skill.

→ **The job this maps to:** ground segment and TT&C; (telemetry, tracking & command) — satellite operators, ground-station networks, and RF/antenna teams.

Auto-Tracking Ground Station — Pointing Chain



Software propagates the orbit from TLEs → computes azimuth and elevation every second
Motors slew the antenna to follow the pass → SDR holds a lock the whole way across

How It Works

- **Predicting where the satellite is:** Every tracked satellite has published orbital elements (a TLE — two-line element set). An SGP4 propagator turns those plus the current time into the satellite's position, which is then converted to an azimuth (compass bearing) and elevation (angle above the horizon) from your exact location.
- **Azimuth-elevation rotator:** Two motors — one for azimuth, one for elevation — aim the antenna at any point in the sky. Geared steppers (or DC motors with position feedback) move the antenna to the commanded az/el and update continuously as the satellite moves.
- **Why a moving antenna at all:** A high-gain, directional antenna has a narrow beam, which is what gives it the gain to hear a weak satellite — but that beam must stay pointed. A fixed antenna trades gain for coverage; a tracker gives you both by following the target.
- **Closing the loop with the SDR:** An SDR at the feed receives the downlink while the rotator tracks. Because the satellite moves fast, you also correct for Doppler shift in software. Accurate pointing plus Doppler correction keep a solid lock across the whole pass.
- **Horizon-to-horizon automation:** The controller waits for a predicted pass, slews to the rise point (AOS), tracks across the sky, and parks at the set point (LOS) — fully automatic. A scheduler can chain multiple satellites across a day, exactly like an operational ground station.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Geared stepper motors	28BYJ-48 + ULN2003 driver (az + el)	2	\$8
RTL-SDR	Downlink receiver	1	\$30
Microcontroller	ESP32 / STM32 (runs SGP4)	1	\$6
Directional antenna	VHF/UHF Yagi (DIY)	1	\$10
Rotator hardware	Bearings, brackets, mast	1	\$10
Heavier motors (opt.)	NEMA17 + driver for a big antenna	1	\$0-15
Power supply	5-12 V	1	\$6

Build Steps

Build the az-el rotator: Construct a two-axis mount: an azimuth base that spins the whole assembly and an elevation axis that tilts the antenna up. Use motors with enough torque to hold the antenna steady, and add end stops or position feedback.

Mount a directional antenna: Build or attach a lightweight VHF/UHF Yagi as the payload. Balance it on the elevation axis so the motor is not fighting gravity. Run the feedline with a service loop so the cable does not bind as the rotator turns.

Run SGP4 on the controller: Load the target satellite's TLE and implement SGP4 (libraries exist for most MCUs). Feed it your location and the current time; verify it outputs az/el that matches a tracking app for the next pass.

Drive the motors to az/el: Map the computed az/el to motor positions and update once a second. Calibrate the azimuth zero to true north and the elevation zero to the horizon. Watch the antenna track a 'virtual' pass before trying a real one.

Track a live pass + receive: Connect the SDR, wait for a real pass, and let the station track from AOS to LOS. Apply Doppler correction in the SDR software and capture the downlink. Film the antenna sweeping the sky on its own — the money shot.

Interview Questions:

- What is a TLE, and how does SGP4 turn it into a pointing direction?
- Why does a high-gain antenna require active tracking while an omni does not?
- How do you convert a satellite's predicted position into azimuth and elevation for your site?
- Why must you correct for Doppler shift during a low-orbit pass?
- How do you calibrate the rotator's azimuth and elevation references?
- What limits your tracking accuracy, and how would you improve it?

PROJECT 3

Magnetorquer Detumble System

Coils that push against Earth's magnetic field to stop a tumbling platform — how a CubeSat stabilizes after launch

Cost: ~\$30

Difficulty:
Intermediate-Advanced

Build Time: 1-2 weekends

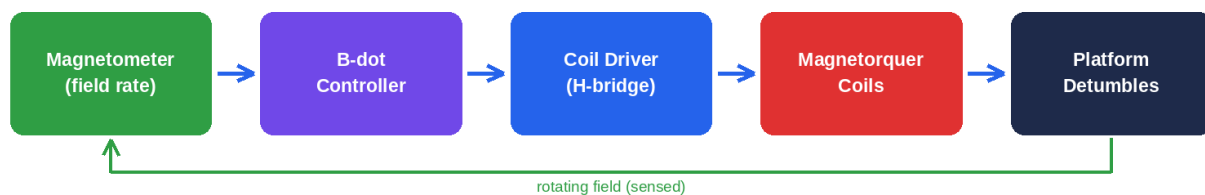
Why This Matters

When a CubeSat is ejected from its deployer it is tumbling, and before it can do anything useful it has to stop spinning and stabilize — a maneuver called detumbling. The clever part: it has nothing to push against, so it pushes against Earth itself. Electromagnets called magnetorquers create a magnetic moment that interacts with Earth's magnetic field to produce a tiny torque, bleeding off the spin. This project builds exactly that: coils on a platform, a magnetometer to sense rotation, and the famous 'B-dot' control law that drives the coils to kill the tumble. It is one of the most elegant ideas in spacecraft engineering — steering against a planet's magnetic field — and it is genuinely buildable and cheap.

What this proves to a recruiter: You understand magnetic actuation, the B-dot detumbling control law, magnetometer sensing and the geomagnetic field, and coil/driver design. Magnetorquers are flown on most CubeSats, and demonstrating detumbling control is a direct, recognizable ADCS skill.

→ **The job this maps to:** spacecraft attitude control (ADCS) — CubeSat and smallsat companies, and ADCS subsystem teams.

Magnetorquer Detumble System — B-dot Control Loop



Magnetometer senses the field changing as the platform spins → B-dot law sets coil currents
Coils torque against Earth's field → spin bleeds off until the platform is stable

How It Works

- **Pushing against a planet:** A magnetorquer is just a coil; running current through it creates a magnetic dipole. That dipole tries to align with Earth's magnetic field — like a compass needle — producing a torque on the spacecraft. With no reaction mass needed, it is an almost-free way to control attitude in low orbit.
- **Sensing the tumble:** A 3-axis magnetometer measures Earth's field in the spacecraft's frame. If the spacecraft is rotating, that measured field vector sweeps around — and the rate at which it changes (its derivative) tells you how fast you are tumbling. This is the key insight behind the control law.
- **The B-dot control law:** B-dot drives each coil proportional to the rate of change of the measured field (the 'B-dot' signal), in the direction that opposes the change. The result is a torque that always fights the tumble, smoothly bleeding off rotational energy until the field stops sweeping. It needs no knowledge of absolute orientation, which is why it is so robust.
- **Driving the coils:** An H-bridge per axis lets the controller push current either direction through each coil, so it can torque whichever way is needed. Coil current sets the dipole strength; the bigger the coil and current, the more torque — traded against power and heat.
- **Demonstrating it on the ground:** Hang the platform on a thin thread (a torsion pendulum) or float it on a bearing, give it a spin, and watch the B-dot controller drive the coils to slow and stop it relative to Earth's field. The same algorithm runs on real CubeSats minutes after deployment.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Magnetometer	3-axis (QMC5883L / LIS3MDL)	1	\$3
Magnetorquer coils	Hand-wound air coils or torque rods (x3)	3	\$8
H-bridge drivers	Per-axis (DRV8833 x2)	2	\$5
Microcontroller	STM32 / ESP32 (B-dot loop)	1	\$6
Battery	LiPo + regulator	1	\$5
Platform	Frame + suspension thread / bearing	1	\$3

Build Steps

Wind the magnetorquers: Wind three air-core coils (or coils on a ferrite rod for more moment) on the three axes of a small frame. More turns and area give more magnetic moment per amp. Measure each coil's resistance and confirm it produces a field with a compass.

Read the magnetometer: Mount the 3-axis magnetometer away from the coils and read all three axes over I2C. Rotate the platform by hand and watch the field vector sweep — confirming you can sense rotation.

Drive coils with H-bridges: Wire each coil to an H-bridge so the MCU can energize it in either polarity. Verify a commanded current produces a deflection near a reference magnet, and that you can reverse the torque direction.

Implement B-dot: Compute the derivative of the measured field, then set each coil's current opposite to that rate of change. Tune the gain so it responds firmly without chattering. Crucially, pulse the coils and read the magnetometer when they are off, so you measure Earth's field, not your own coils.

Detumble on a thread: Suspend the platform on a thin thread or float it on a bearing, spin it up, and switch on B-dot. Watch the rotation slow and settle. Log angular rate vs. time to show the exponential detumble — the exact curve a CubeSat produces in orbit.

Interview Questions:

- How does a magnetorquer produce torque without any reaction mass?
- What does the magnetometer's rate of change tell you about the spacecraft's motion?
- Explain the B-dot control law and why it does not need absolute attitude knowledge.
- Why must you sense the field with the coils de-energized?
- What are the limitations of magnetic actuation — when can't it control an axis?
- How do magnetorquers complement reaction wheels in a full ADCS?

PROJECT 4

Satellite-in-a-Box Flight Computer

An avionics stack that reads its sensors and beams CCSDS-style telemetry packets down to a ground dashboard

Cost: ~\$50

Difficulty:
Intermediate-Advanced

Build Time: 2-3 weekends

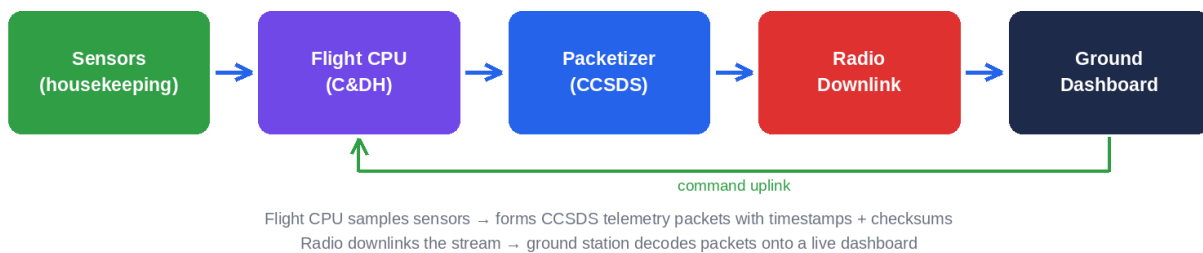
Why This Matters

Every spacecraft has a brain: the command-and-data-handling (C&DH;) computer that reads the sensors, runs the housekeeping, packages everything into telemetry, and beams it to the ground while accepting commands back up. This project builds a working one — a flight-computer stack that samples its sensors, forms proper telemetry packets, transmits them over a radio link to a ground station you also build, and decodes them onto a live dashboard. Use the real spacecraft data standard (CCSDS) for the packets and you have built the same data architecture that flies on actual missions. It is less flashy than a thruster, but it is the literal nervous system of a satellite — and the flight-software skill is in high demand.

What this proves to a recruiter: You understand command-and-data-handling architecture, telemetry packetization (CCSDS), reliable serial/radio links, and ground-side decoding and display. Flight software and C&DH; are central to every space mission, and a working telemetry pipeline is a standout systems project.

→ **The job this maps to:** command & data handling (C&DH;) and flight software — satellite primes, CubeSat companies, and avionics teams.

Satellite-in-a-Box Flight Computer — Telemetry Pipeline



How It Works

- **The C&DH; role:** The flight computer is the spacecraft's housekeeper: on a fixed schedule it reads every sensor (voltages, currents, temperatures, attitude), checks system health, stores data, and assembles telemetry for downlink — all while listening for commands from the ground. It is the integration point for every other subsystem.
- **Housekeeping telemetry:** Your stack samples a set of 'housekeeping' channels — battery voltage, board temperature, IMU attitude, a packet counter — at a steady rate. This is exactly the data operators watch to know a real satellite is alive and healthy.
- **CCSDS packetization:** Spacecraft do not send raw numbers; they wrap data in standardized packets. The CCSDS space-packet format adds a header (identifying the data type, a sequence count, a timestamp) and a checksum. Implementing it means your telemetry is structured, parseable, and corruption-checkable — the way flight data really works.
- **The radio link:** A simple radio module (a UHF data module, or LoRa for easy range) carries the packet stream to the ground. You frame the packets for the link, and the ground side detects frame boundaries, validates checksums, and rejects corrupted packets — the reliability layer every downlink needs.
- **Ground decoding + dashboard:** On the ground, software receives the stream, parses the CCSDS packets back into named values, and plots them live — voltages trending, temperature rising, attitude changing. Add a command uplink and you can send instructions back, completing a real telemetry-and-command loop.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Flight MCU	STM32 (C&DH; computer)	1	\$6
Sensor suite	IMU + temp + INA voltage/current	1	\$8
Radio modules	UHF data / LoRa pair (flight + ground)	2	\$16
SD card module	Onboard data storage	1	\$3
Ground computer	Raspberry Pi / laptop (dashboard)	1	—
USB-serial bridge	Ground radio to PC	1	\$3
Power + battery	Regulator + LiPo	1	\$6
Protoboard + wiring	Stack assembly	1	\$5

Build Steps

Bring up the sensors: Wire the IMU, a temperature sensor, and a voltage/current monitor to the flight MCU. Sample them on a fixed schedule (say 1 Hz) and confirm clean readings. These are your housekeeping channels.

Build CCSDS packets: Implement the CCSDS space-packet structure: a primary header with an application ID, a sequence counter, and a timestamp, then your sensor payload, then a checksum. Print the raw bytes and verify the structure by hand before transmitting.

Add the radio downlink: Connect a radio module and frame the packets for it (sync word + length + packet + CRC). Transmit at a steady rate. On the second radio at the ground, capture the framed bytes and confirm you are receiving the stream.

Decode on the ground: Write ground software that finds frame boundaries, validates the CRC, parses the CCSDS header and payload, and prints named telemetry. Reject and count corrupted packets so you can see link quality.

Build the dashboard + uplink: Plot the telemetry live (a simple web dashboard or plotting script) — voltage, temperature, attitude, packet count. Add a command uplink so the ground can change a setting on the flight computer. Demonstrate the full loop end to end.

Interview Questions:

- What does the command-and-data-handling subsystem do on a spacecraft?
- Why use a standardized packet format like CCSDS instead of sending raw values?
- What is in a CCSDS primary header, and what is the sequence count for?
- How does the ground side detect and reject corrupted telemetry?
- Why timestamp telemetry at the source rather than on receipt?
- How would you make the downlink robust to dropped packets and bit errors?

PROJECT 5

Sun-Vector Sensor

A photodiode-array sensor that computes the exact direction to the Sun — the most-flown attitude sensor in space

Cost: ~\$25

Difficulty: Intermediate

Build Time: 1 weekend

Why This Matters

To point itself, a spacecraft first has to know which way it is facing — and the simplest, cheapest, most-flown way to get a reference is to find the Sun. A sun sensor measures the direction to the Sun in the spacecraft's own frame, and that single vector is enough to constrain attitude for power-positioning and coarse pointing. This project builds a real one: an arrangement of photodiodes (or a pinhole over a small sensor array) whose relative readings encode the Sun's angle, plus the math to turn those readings into a sun vector. It is a clean, satisfying sensor project — analog front end, geometry, and calibration — and it is a genuine flight sensor, not a toy.

What this proves to a recruiter: You understand optical attitude sensing, photodiode analog front-ends and ratiometric measurement, the geometry of converting intensities into an angle, and sensor calibration. Sun sensors fly on nearly every satellite, and building one is a recognizable ADCS sensing skill.

→ **The job this maps to:** attitude determination (ADCS) — CubeSat and satellite companies, and sensor/ADCS subsystem teams.

Sun-Vector Sensor — Angle Determination



Sunlight hits photodiodes at different angles → each produces a different photocurrent
Ratios of the readings encode the Sun's direction → MCU computes the sun vector

How It Works

- **Why the Sun:** The Sun is a bright, effectively point source in a known direction, making it the easiest celestial reference to measure. Knowing the sun vector in the body frame lets a spacecraft orient its solar panels and perform coarse attitude control — which is why almost every satellite carries a sun sensor.
- **Photodiodes at angles:** If you mount photodiodes facing slightly different directions (or put a single pinhole/mask over a small array), each receives sunlight at a different incidence angle. Since a photodiode's output falls off predictably with angle (roughly as the cosine), the relative outputs depend on where the Sun is.
- **Ratiometric measurement:** Absolute brightness varies with distance and conditions, so you do not use raw values — you use ratios between diodes. Ratios cancel overall intensity and depend only on the angle, which is the trick that makes the sensor robust. This is the same principle as commercial analog sun sensors.
- **Analog front end:** Each photodiode's tiny current is converted to a voltage by a transimpedance amplifier and read by the MCU's ADC. Matching the channels and keeping noise low matters, because the angle accuracy comes directly from how cleanly you measure the photocurrents.
- **Computing the sun vector:** From the calibrated ratios, the MCU computes the two angles that define the Sun's direction (azimuth and elevation in the sensor frame), outputting a unit vector. Calibrating against known Sun angles turns the raw geometry into an accurate sensor.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Photodiodes	Matched (x4) or a quad photodiode	4	\$6
Op-amps	Transimpedance amplifiers (quad)	1	\$3
Pinhole / mask + housing	Defines incidence geometry	1	\$4
Microcontroller	With multi-channel ADC	1	\$5
Precision resistors	TIA feedback, matched	1	\$2
OLED / output	Show the computed angle	1	\$4

Build Steps

Arrange the photodiodes: Either tilt four photodiodes outward in a shallow pyramid, or place a pinhole mask above a quad photodiode so the light spot shifts with Sun angle. Both encode direction into the relative outputs; the pinhole approach is more precise.

Build the analog front end: Convert each photodiode current to a voltage with a transimpedance amplifier using matched feedback resistors. Read all channels on the MCU's ADC. In shade, confirm near-zero; in sun, confirm strong, sensible readings.

Take ratios, not raw values: Compute ratios between opposing channels so overall brightness cancels. Verify that moving a light source (or tilting the sensor toward the Sun) changes the ratios smoothly and repeatably while brightness changes do not.

Calibrate against known angles: Point the sensor at the Sun (or a collimated lamp) at several known angles and record the ratios. Fit a simple model mapping ratios to angle — this calibration is what turns the device into a real sensor.

Output the sun vector: Use the calibrated model to output the Sun's azimuth and elevation, or a unit vector, live on the OLED. Demonstrate it tracking the Sun's direction as you rotate the sensor — and discuss its accuracy and field of view.

Interview Questions:

- Why is the Sun the most common attitude reference for spacecraft?
- Why use ratios of photodiode outputs instead of absolute intensities?
- How does a photodiode's output depend on the angle of incoming light?
- What does the transimpedance amplifier do, and why does channel matching matter?
- How do you calibrate raw readings into an accurate angle?
- What are a sun sensor's limitations (eclipse, albedo, field of view)?

PROJECT 6

CubeSat Power System (EPS)

Solar input, battery management, regulated distribution, and power telemetry — a real satellite's electrical heart

Cost: ~\$45

Difficulty:
Intermediate-Advanced

Build Time: 2 weekends

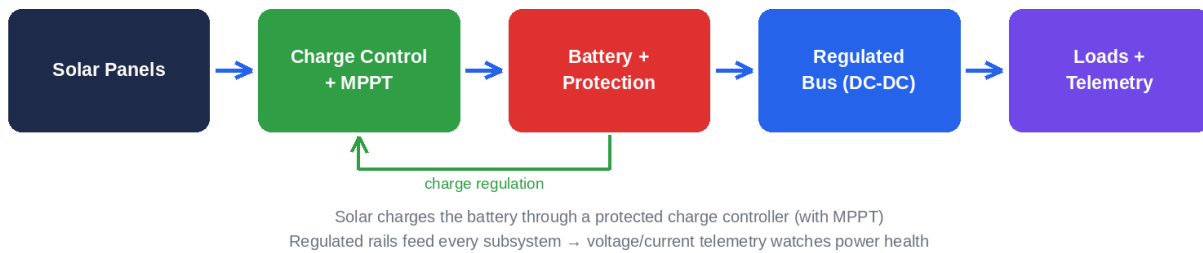
Why This Matters

A satellite lives and dies by its power system. The EPS (electrical power subsystem) harvests energy from solar panels, charges and protects the battery, regulates it into the voltage rails every other subsystem needs, and reports power health as telemetry — all of it autonomously, because there is no one to flip a switch in orbit. This project builds a real one: a solar input stage that charges a lithium battery safely, regulated output rails, protection against over-current and over-discharge, and voltage/current telemetry on every channel. It is the same architecture as a flight EPS, and it is the subsystem that keeps everything else alive — which is exactly why power engineers are always in demand.

What this proves to a recruiter: You understand solar energy harvesting, lithium battery charging and protection, DC-DC regulation and power distribution, and power telemetry/monitoring. The EPS is mission-critical on every spacecraft, and building one demonstrates real power-electronics and systems competence.

→ **The job this maps to:** electrical power systems (EPS) — satellite and CubeSat companies, and power-electronics subsystem teams.

CubeSat Power System (EPS) — Power Architecture



How It Works

- **Harvesting solar power:** Solar cells are the only energy source in orbit, but their output swings with sun angle and temperature. The charge stage (often with MPPT, as in the solar-controller project) extracts as much power as conditions allow and uses it to charge the battery.
- **Battery charging + protection:** A lithium battery must be charged within strict voltage and current limits and protected against over-charge, over-discharge, over-current, and temperature extremes — any of which can damage or destroy it. The EPS enforces all of this automatically; in space there is no replacing a cooked battery.
- **Regulated distribution:** Different subsystems need different, clean voltages (e.g. 3.3 V for logic, 5 V for radios). DC-DC converters generate these regulated rails from the battery, and the EPS distributes them — often with the ability to switch individual loads on and off.
- **Protection and autonomy:** Each rail has over-current protection so one faulty subsystem cannot pull down the whole bus, and the battery has an under-voltage cutoff so the satellite can safely 'brown out' and recover. This autonomy is essential — the spacecraft must protect itself.
- **Power telemetry:** The EPS measures voltage and current at the solar input, the battery, and each rail, and reports them as telemetry. Operators watch this to understand the energy balance — is the satellite generating more than it consumes? — one of the most important health metrics of any mission.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Solar panel(s)	Small PV panel / cells	1	\$10
Charge controller	Li-ion charger IC (+ MPPT input)	1	\$5
Battery + protection	Li-ion cell + protection board	1	\$8
DC-DC regulators	3.3 V + 5 V buck modules	2	\$6
Voltage/current monitors	INA219 / INA226 (x3)	3	\$6
Load switches	MOSFET high-side switches	1	\$3
Microcontroller	Reads telemetry, controls switches	1	\$5
Protoboard + wiring	Assembly	1	\$2

Build Steps

Build the charge stage: Wire the solar panel to a lithium charge-controller IC sized for your cell, ideally with an MPPT input stage. Confirm it charges the battery from sunlight within the correct voltage/current limits and stops at full charge.

Add battery protection: Use a protected cell or add a protection board enforcing over-charge, over-discharge, and over-current limits. Test the under-voltage cutoff by loading the battery down and confirming it disconnects safely.

Generate regulated rails: Add buck converters for 3.3 V and 5 V from the battery. Verify each rail is clean and holds voltage under load. These are the rails the rest of your 'satellite' would run on.

Instrument every channel: Place INA-class voltage/current monitors on the solar input, the battery, and each rail, and read them on the MCU. Now you can see exactly where power is flowing and how much.

Add switching + telemetry: Add high-side load switches so the MCU can turn rails on and off, and stream all the power telemetry (with over-current protection) to a display or your flight computer. Demonstrate the energy balance: solar in vs. load out, charging and discharging.

Interview Questions:

- What are the four main jobs of a spacecraft EPS?
- Why must lithium batteries be so carefully protected, especially in space?
- Why does each power rail need its own over-current protection?
- What is the energy balance of a satellite, and why do operators watch it?
- Why include MPPT in the solar charge stage?
- How would you design the EPS to survive and recover from a brown-out?

Six subsystems. One satellite.

Build them and you've built the real thing.

Propulsion, attitude control, attitude sensing, flight software, ground systems, and power. Put these six together and you do not have six gadgets — you have the functional subsystems of an actual satellite, built on a bench for the price of a textbook each. A thruster that slews a platform, coils that detumble against Earth's field, a sun sensor, a flight computer streaming telemetry, a tracking ground station, and a power system to keep it all alive. That is genuinely how spacecraft are organized, and how space companies hire.

The space industry recruits by subsystem — GNC, ADCS, propulsion, C&DH;, ground, EPS — and a working demo of any one of these speaks that language directly. You are not claiming to have built a satellite; you are showing you can build the part they need.

Build even one of these. Log the slew, plot the detumble curve, screenshot the telemetry dashboard, show the energy balance. Put it on your GitHub with a proper README and the subsystem it maps to. You will have a portfolio piece that puts you in the room with a real space-hardware team.

Follow @1nska on Instagram for build logs, new guides, and projects that build real spacecraft.

— Enes (@1nska) • Electrical Engineer